LIMR: Intent-Aware Mashup API Recommendation via LLM-Augmented Multi-Scale Fusion

Yao Zhang, Yude Bai, Minhong Dong, Keqing Cen, Ji Zhang, Qiang Hu, Wei Ma, Yongqiang Lyu, Ruitao Feng, Xiaohong Li, Junjie Wang, Lingxiao Jiang, Yang Liu

***Abstract*—The increasing availability of Web APIs has amplified the complexity of mashup creation, where developers must identify compatible and functionally relevant APIs based on often ambiguous natural language descriptions. Traditional methods also fall short in capturing hierarchical semantic cues, modeling compatibility, and aligning with developer intent. Although large language models (LLMs) offer strong generalization capabilities, they remain unreliable in mashup recommendation due to hallucinated outputs, limited controllability, and token-length constraints when dealing with large-scale API repositories.**

To overcome these limitations, we introduce *LIMR*, an intent-aware mashup recommendation framework that combines LLM-augmented semantic reasoning with structured, multi-scale neural modeling. *LIMR* first prompts a LLM to extract high-level intent from user requirements, which serves as a global semantic signal. This intent is fused with low-level, multi-scale features extracted by a convolutional encoder, which are designed to capture fine-grained lexical/phrasal patterns at different granularities and provide precise semantic grounding for API matching. These heterogeneous representations are further contextually refined through a Transformer-based interaction module. To handle nonlinear semantic dependencies and compositional complexity, *LIMR* integrates a Kolmogorov–Arnold Network (KAN) with learnable activation functions, enhancing the model's capacity to capture intricate feature interactions. The entire framework is optimized via LLM, incorporating auxiliary objectives such as mashup category prediction and API quality estimation to guide generalization and reduce overfitting.

Comprehensive experiments on the ProgrammableWeb and APIBench datasets show that *LIMR* significantly outperforms state-of-the-art baselines, which the ranking-oriented metrics, including NDCG and mAP, achieves improvements of 17.1%–34.2% over the strongest competitors. These results confirm the effectiveness of *LIMR* 's hybrid design in delivering precise, robust, and intent-aware mashup API recommendations, especially in scenarios where LLMs alone fail to meet accuracy and scalability demands.

***Index Terms*—Mashup recommendation, Multi-Scale model fusion, Large Language model, automated labeling.**

Yao Zhang, Minhong Dong, Qiang Hu, Yongqiang Lyu, Xiaohong Li, Junjie Wang are with School of Cybersecurity of Tianjin University, China. E-mail: zzyy@tju.edu.cn, dongminhong@tju.edu.cn, qianghu@tju.edu.cn, lyuyq@tju.edu.cn, xiaohongli@tju.edu.cn, junjie.wang@tju.edu.cn.

Yude Bai are with School of Software, Tiangong University, Tianjin, China. E-mail:baiyude@tiangong.edu.cn.

Keqing Cen is with Guangxi Guiguan Electric Power Co.,Ltd. E-mail: lubincen1130@gmail.com.

Ji Zhang is with the University of Southern Queensland, Australia. E-mail: Ji.Zhang@unisq.edu.au.

Wei Ma and Lingxiao Jiang are with the Singapore Management University. E-mail:weima@smu.edu.sg, lxjiang@smu.edu.sg.

Ruitao Feng is with the Southern Cross University. E-mail:ruitao.feng@scu.edu.au.

Yang Liu is with the School of Computer Science and Engineering, Nanyang Technological University, Singapore. E-mail:yangliu@ntu.edu.sg.

Qiang Hu, Wei Ma, and Xiaohong Li are the corresponding authors.

I. Introduction

In recent years, the number of publicly accessible APIs has increased at an unprecedented rate, offering developers access to a broad spectrum of services, including payments, geolocation, and social media, *etc.*. To facilitate rapid innovation and efficient application development, developers increasingly construct mashups, the applications that integrate and coordinate multiple APIs from different providers to deliver composite functionalities that individual APIs cannot support alone. Platforms such as MuleSoft [1] have emerged to support this development by providing robust tools for API discovery, integration, and service orchestration.

However, as software systems grow increasingly complex and API ecosystems expand, developers face growing difficulties in discovering and composing APIs into effective mashups. These challenges arise from the vast and ever-growing number of available APIs (*e.g.*, more than 19,000 on MuleSoft), the lack of standardized and semantically rich descriptions, and the difficulty of ensuring compatibility among APIs in terms of functionality, data formats, and invocation logic. Moreover, user requirements are frequently vague or articulated in natural language, which introduces ambiguity and complicates the process of generating precise and contextually appropriate API compositions.

Traditional mashup recommendation methods primarily rely on content-based filtering strategies [2]–[4], which attempt to match mashup requirements with APIs by comparing their textual or structural representations. These methods often utilize techniques such as keyword matching, vector space models, or semantic embeddings to measure similarity between the natural language descriptions of mashups and the functional descriptions of APIs. Some approaches also incorporate syntactic structures or metadata features to improve alignment. However, the effectiveness of content-based approaches is constrained by the quality and consistency of textual information, which is often sparse, noisy, or poorly structured in real-world platforms. As a result, these methods struggle to generalize beyond explicitly observed patterns and tend to fail in scenarios involving vague or semantically ambiguous user requirements. In contrast, collaborative filtering techniques [4]–[6] recommend APIs based on historical co-usage patterns between mashups and services. Typically, these methods construct a mashup-API interaction matrix and apply algorithms such as matrix factorization or neighborhood-based models to infer latent associations. While collaborative filtering can uncover implicit usage trends, it lacks the capacity to understand

the semantic content of user requests or API functionalities. Consequently, it performs poorly when addressing new or infrequent mashup requirements, particularly those expressed in informal or domain-specific language.

Traditional mashup recommendation methods have also used small language models (SLMs) such as TF-IDF [7], Word2Vec [8], [9], and BERT [10], [11] variants to extract statistical patterns and shallow semantic features from API descriptions. However, these SLM-based approaches process requirements through statistical pattern matching, which limits their ability to capture implicit user intent. As a result, they struggle to interpret vague or ambiguous user requirements because their limited model capacity restricts understanding of complex API relationships.

Recent advancements in large language models (LLMs) have opened new opportunities for understanding natural language and code representations. Some approaches directly leverage LLMs to recommend relevant APIs based on user intent or contextual information [12]–[14]. While LLMs offer superior natural language generalization compared to SLMs, they face different challenges in mashup recommendation tasks. Most methods are limited to single API recommendation, whereas mashup recommendation requires modeling complex relationships such as functional synergy, parameter dependencies, and invocation order among multiple APIs. Furthermore, LLMs tend to generate inaccurate or nonexistent APIs due to hallucination and semantic drift. Although Retrieval-Augmented Generation (RAG) attempts to constrain the reasoning space by grounding LLMs in external knowledge bases, the sheer number of available APIs (for example, over 19,000 APIs on MuleSoft Platform) can easily exceed token limits, which prevents effective handling of large-scale API repositories.

To address these limitations, we propose *LIMR* (**L**LM-based **I**ntent-aware **M**ashup **R**ecommendation), a hybrid framework that combines LLMs for semantic understanding with lightweight neural architectures for structured reasoning to achieve both semantic depth and technical precision. Our approach works in two main phases. In the first phase, LIMR leverages an LLM as a semantic reasoning engine to analyze natural language requirements and extract high-level functional intents. Unlike direct API generation, the LLM serves as an intent classifier, identifying abstract functional categories (e.g., "payment processing," "geographic services," "data visualization") that capture the user's true needs. This design choice addresses the hallucination problem by constraining the LLM to produce structured intent labels rather than specific API names, transforming its broad semantic understanding into actionable guidance. The second phase employs a sophisticated neural architecture that translates these intent signals into precise API recommendations. This architecture comprises three components: (1) A multi-scale Convolutional Neural Network (CNN) encoder that processes the original requirement text to capture both fine-grained keywords and broader phrasal patterns essential for API matching; (2) A Transformer-based context modeler that integrates the LLM-derived intents with the CNN features through multi-head attention to align local textual patterns with global functional understanding; (3) A Kolmogorov-Arnold Network (KAN) that learns complex decision boundaries between similar APIs that traditional Multi-Layer Perceptron (MLPs) struggle to distinguish.

Through this hybrid design, LIMR address the limitations of both paradigms: it achieves the semantic sophistication needed to interpret complex, ambiguous requirements while maintaining the factual grounding and computational efficiency required for practical deployment. The framework effectively transforms the mashup recommendation problem from pure similarity matching to intent-guided search, which generates recommendations that are both functionally appropriate and technically valid.

In summary, this paper makes the following important contributions:

- To our best knowledge, we are the first to formalize intent-aware mashup recommendation as a multi-level semantic alignment problem, and introduce a theoretical framework that bridges high-level user intent with low-level API functionality.
- We propose a novel architectural paradigm that synergistically combines LLM-derived semantic understanding with lightweight neural recommendation models. This design principle addresses a fundamental trade-off in recommendation systems, which balances semantic richness with computational efficiency and precision. Our multi-scale fusion mechanism also establishes a new methodology for handling heterogeneous semantic representations in recommendation tasks.
- We develop a theoretical framework for modeling API compatibility that goes beyond traditional similarity metrics, introducing compositional coherence and functional complementarity as explicit optimization objectives. This helps ensure that recommended API sets not only match user requirements individually but also work together effectively as a cohesive mashup.
- Comprehensive experiments on the ProgrammableWeb and APIBench datasets demonstrate that *LIMR* achieves substantial improvements over state-of-the-art baselines, with gains of 17.1%-34.2% across all the metrics. Case studies on real-world mashups further validate the framework's practical effectiveness.

The rest of this paper is organized as follows. Section II introduces the preliminaries, including motivating examples, formal problem definitions, and the key challenges. Section III provides an overview of the proposed LIMR framework and presents its core design principles. Section IV presents the methodology in detail, describing both the intent-oriented semantic encoding and the contextual modeling components. Section V reports the experimental evaluation, including research questions, datasets, baselines, performance comparisons, ablation studies, and real-world case analyses. Section VI reviews related work in API and mashup recommendation. Finally, Section VII concludes the paper and outlines potential directions for future research.

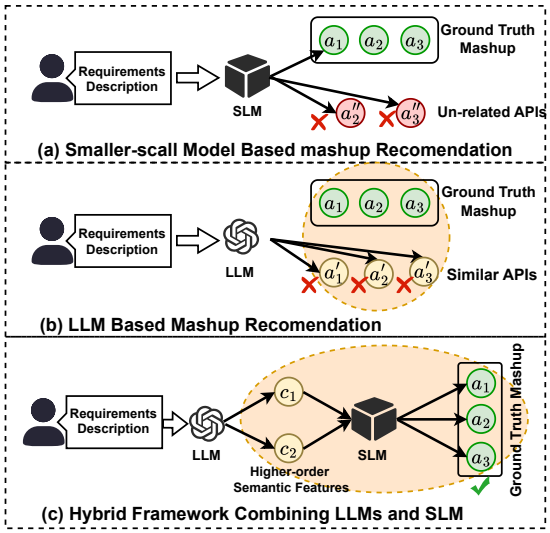


Fig. 1: Conceptual comparison of mashup recommendation approaches. (a) SLM-based methods produce unrelated APIs due to shallow semantic matching. (b) LLM-based methods generate functionally similar but non-existent APIs. (c) Our hybrid approach (LIMR) leverages LLM-extracted intents to guide accurate API selection.

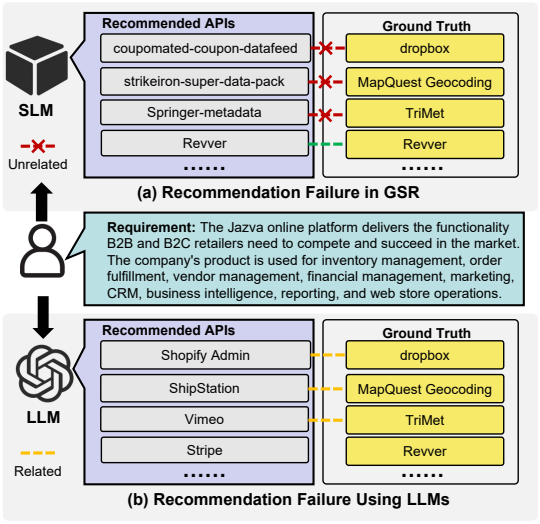
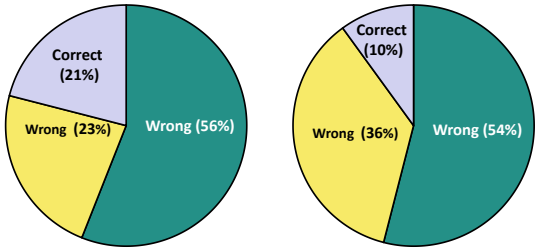


Fig. 2: Case study: API recommendations for Jazva e-commerce platform



(a) Results on GPT-5 (b) Results on DeepSeek

Fig. 3: Top-10 Accuracy Across the LLMs

II. Preliminaries

A. Motivating Example

To illustrate the limitations of existing approaches and motivate our hybrid framework, we examine how different recommendation paradigms handle real-world mashup scenarios. Figure 1 provides a conceptual comparison of three approaches, while Figure 2 demonstrates these failures with a concrete example from the ProgrammableWeb dataset.

Figure 1(a) illustrates the fundamental problem with traditional small language model (SLM) approaches. When presented with a user requirement, SLMs recommend APIs (a_1'', a_2'', a_3'') where a_2'' and a_3'' appear semantically similar to the ground truth (a_1, a_2, a_3) but are functionally unrelated. This pattern manifests concretely in Figure 2(a), where the generative service recommendation approach (GSR) method [15] attempts to recommend APIs for the Jazva platform, a comprehensive B2B/B2C retail system requiring inventory management, order fulfillment, and logistics capabilities. The ground truth APIs dropbox, MapQuest Geocoding, TriMet, Revver provide essential services, which are cloud storage, location services, transit routing, and video hosting, respectively. However, GSR recommends coupomated-coupon-datafeed, strikeiron-super-data-pack, Springer-metadata, Revver. While it correctly identifies Revver (marked with a green checkmark), the other three APIs are completely unrelated (marked with red X's). The coupon datafeed targets promotional content rather than core operations, strikeiron provides generic data services without retail specificity, and Springer-metadata serves academic publishing. This failure exemplifies the conceptual problem shown in Figure 1(a): surface-level keyword matching ("data," "management") without understanding that the platform specifically needs geographic and transit APIs for delivery optimization.

Figure 1(b) depicts a different failure mode with LLM-based approaches, where recommended APIs (a_1', a_2', a_3') fall within functionally similar regions (shown as orange areas) but do not match the actual ground truth. Figure 2(b) confirms this pattern empirically: ChatGPT-4.0 recommends Shopify Admin, ShipStation, Vimeo, Stripe for the same Jazva requirement. These APIs demonstrate correct functional understanding, but fail on technical accuracy (marked with orange dashed lines indicating "Related" but incorrect). The critical issue is that "Shopify Admin" refers to a proprietary dashboard rather than an accessible API, revealing how LLMs generate plausible-sounding but non-existent services. While ShipStation and Stripe provide relevant e-commerce functionality, they cannot substitute for the specific geographic services (MapQuest Geocoding) and transit information (TriMet) essential for logistics. This shows the hallucination problem in Figure 1(b) that LLMs understand intent but lack grounding in the actual API repository.

We further evaluate the feasibility of directly using LLMs for API recommendation on a random sample of 100 mashup descriptions. Each top-10 output is categorized as Correct, Partially Correct, or Wrong. As shown in Figure 3, the fully correct rate is only 10%–21% (GPT-5: 21%, DeepSeek: 10%),

while Wrong cases dominate (54%–56%). This indicates that naive LLM-based recommendation is still unreliable, motivating more structured modeling beyond direct prompting.

These failure patterns motivate our hybrid approach LIMR shown in Figure 1(c). The key insight is to leverage LLMs to extract high-level intent categories c_1', c_2' that capture the functional requirements in the Jazva case. These intent labels enhance the original semantic representations, guiding a smaller model to search within functionally appropriate API subspaces while maintaining access to the actual API catalog. For the Jazva example, LIMR successfully identifies all ground truth APIs. The LLM-derived intents help recognize that dropbox serves document management needs, MapQuest Geocoding enables location-based services, TriMet provides transit routing for deliveries, and Revver supports video-based product demonstrations. LIMR combines the deep semantic understanding of LLMs with the efficiency and generalizability of small models, enabling more accurate and purpose-driven API composition even in complex and ambiguous scenarios.

B. Definitions

Definition 1 (User Requirement): Let $\mathcal{V}$ be the tokenizer vocabulary. A user requirement is a token sequence $R = (w_1, \ldots, w_n)$ with $w_i \in \mathcal{V}$ and variable length n, which is a natural language description expressing the functional need of the user for composing a mashup.

Definition 2 (Intent Label): Let $\mathbf{C} = \{c_1, c_2, \ldots, c_m\}$ be a predefined set of m semantic intent categories (e.g., "payment processing," "geographic services," "data storage"), where each c_i represents a functional capability. Given requirement $\mathbf{R}$, an LLM-based extraction function produces a set of intent labels $\mathbf{L} \subseteq \mathbf{C}$ that captures the high-level functional goals implied in $\mathbf{R}$. These labels serve as semantic constraints to guide API selection.

Definition 3 (Enriched Representation): Let φ be a representation function that fuses the user requirement $\mathbf{R}$ with the predicted intent label L to form an enriched representation $\varphi(\mathbf{R}, \mathbf{L}) \in \mathbb{R}^d$, where d is the embedding dimension.

$$\varphi(\mathcal{R}, \mathcal{L}) \triangleq \mathrm{Concat}(\chi(R), g(L)) = \mathrm{Concat}(U_{\text{MSE}}, T_{\text{semantic}}) \tag{1}$$

where $\chi(R) = U_{\text{MSE}}$ is the multi-scale semantic representation extracted from the requirement text, and $g(L) = T_{\text{semantic}}$ maps the categorical intent label(s) to a continuous vector.

C. Problem Formulations:

1) Mashup Recommendation: Let $\mathbf{A} = \{a_1, a_2, \ldots, a_n\}$ represent the API repository containing n available APIs, where each a_i is described by its functional metadata and textual description. The intent-aware mashup recommendation task learns a prediction function $f : (\mathbf{R}, \mathbf{L}) \to \hat{\mathbf{A}}$, where f maps the user requirement $\mathbf{R}$ and its extracted intent labels $\mathbf{L}$ to a ranked subset $\hat{\mathbf{A}} \subseteq \mathbf{A}$ of APIs that collectively fulfill $\mathbf{R}$'s functional requirements.

2) Formal rationale for intent-awareness: We model the intent as a latent high-level semantic variable that captures the functional goal(s) implied by $\mathbf{R}$. Let Y denote the supervision signal used for learning (e.g., the relevance label or ranking score of a candidate API). We use $\Pr(\cdot)$ to denote probabilities in the following analysis. When intent is not explicitly modeled, the true intent variable is effectively unobserved, and a recommender trained only on $\mathbf{R}$ corresponds to learning a conditional mixture over possible intent modes:

$$\Pr(Y \mid \mathbf{R}) = \sum_{l} \Pr(Y \mid \mathbf{R}, l) \Pr(l \mid \mathbf{R}), \tag{2}$$

which indicates that, when a requirement $\mathbf{R}$ can correspond to multiple intents with overlapping surface text, prediction based solely on $\mathbf{R}$ aggregates (and thus may blur) intent-specific decision boundaries, increasing conditional uncertainty. In contrast, if the intent variable is informative about the target given the requirement (i.e., $I(Y; \mathbf{L} \mid \mathbf{R}) > 0$), conditioning on $\mathbf{L}$ reduces the conditional uncertainty:

$$H(Y \mid \mathbf{R}, \mathbf{L}) \leq H(Y \mid \mathbf{R}), \tag{3}$$

and therefore can reduce the Bayes-optimal expected risk. This provides a principled justification for incorporating LLM-extracted intents as additional semantic conditions for mashup recommendation.

3) Robustness to imperfect intent extraction: In practice, we observe $\hat{\mathbf{L}}$ rather than the (unobserved) true intent $\mathbf{L}$. Let $\Pr(\hat{\mathbf{L}} = \mathbf{L}) = 1 - \epsilon$ denote the intent extraction accuracy, where $\epsilon \in [0, 1]$ is the intent extraction error rate . When ϵ is small, conditioning on $\hat{\mathbf{L}}$ preserves most of the disambiguation benefit of intent conditioning, while the impact of intent noise is controlled by the error rate and the maximal discrepancy between intent-conditioned predictors across intent modes. This motivates using LLM-extracted intents as soft semantic constraints, which we further validate empirically via ablation and comparative experiments.

4) Optimization Objective.: Given a training set $\mathcal{D} = \{(R_i, L_i, A_i)\}_{i=1}^{N}$, our goal is to learn a predictor that assigns higher scores to the ground-truth APIs in A_i than to non-relevant APIs. We train the model by minimizing the loss $\mathcal{L}$ (defined in Equation 20-21). At test time, candidate APIs are ranked by the predicted scores and we report top-K metrics (e.g., Precision@K, Recall@K, and NDCG@K in Section V.

D. Challenges for Our Hybrid Approach

To leverage the strength of both large and small language models in the mashup recommendation task, we need to overcome the following challenges:

- **Challenge 1: User Intent Extraction and Feature Fusion.** Given the flexibility and variability of natural language queries, designing stable and effective prompts is critical to ensuring that LLM can accurately and robustly map diverse and ambiguous user expressions to predefined intent categories. This remains a significant challenge due to the inherently unstructured and highly variable nature of user inputs. In addition, intents inferred by LLMs are often abstract and high-level, whereas the features extracted by underlying encoders are typically low-level and multi-scale. This semantic and structural gap results in a heterogeneous representation space,

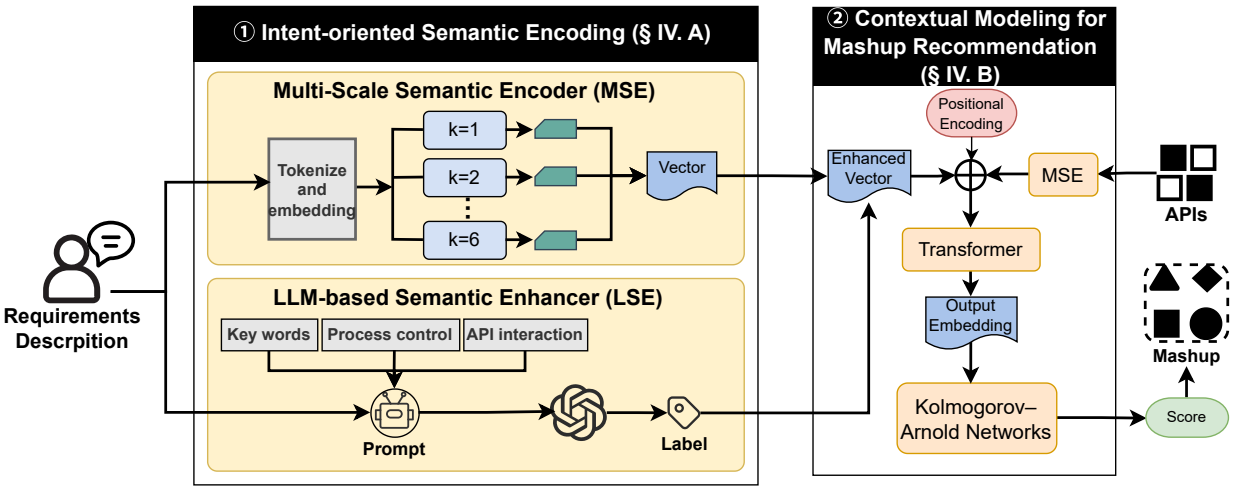


Fig. 4: An overview of the *LIMR* architecture: Intent-oriented semantic encoding via LLM and multi-scale CNN (left) integrated with contextual modeling through Transformer and KAN (right) for mashup recommendation.

where effective fusion of these features becomes particularly challenging. Addressing this fusion problem is essential for enabling context-aware and precise mashup recommendations.

- **Challenge 2: Contextual Modeling for Heterogeneous Semantic Fusion Features.** The high-level intent semantics extracted by LLMs and the multi-scale semantics generated by low-level encoders exhibit significant differences in granularity and structure. Directly fusing these representations often results in a complex feature space characterized by inconsistent dimensions and heterogeneous semantic hierarchies. Accurately modeling semantic dependencies while avoiding information interference within this heterogeneous semantic space remains a key obstacle to improving recommendation accuracy.

III. Overview

To effectively model the latent user intentions and fine-grained semantics embedded in user natural language descriptions, this paper proposes an **L**LM based **I**ntent-aware **M**ashup **R**ecommendation framework (*LIMR*). Figure 4 illustrates the overall architecture of *LIMR*, which addresses the key challenges through two synergistic modules. (1) **Intent-Oriented Semantic Encoding addresses Challenge 1** by extracting and integrating multi-level semantic representations. The module employs a dual-encoder strategy: (i) an LLM-based Semantic Enhancer (LSE) that analyzes the requirement through task-specific prompts to extract high-level functional intents, transforming ambiguous natural language into structured semantic categories; and (ii) a Multi-Scale Semantic Encoder (MSE) that applies parallel CNN filters to capture fine-grained textual patterns from character-level features to phrasal contexts. This dual encoding creates an enriched representation that aligns API selection with user intent. (2) **Contextual Modeling for Mashup Recommendation tackles Challenge 2** through a Transformer+KAN architecture that learns complex mappings from the heterogeneous feature space , capturing cross-source dependencies while mitigating information interference. The Transformer component models global dependencies through multi-head self-attention, enabling the model to align intent signals with local semantic cues across the entire requirement context. To prevent contextualized signals from overriding the original evidence, we further employ a residual weighted fusion mechanism, which injects dependency-aware information while preserving the fused semantic features. Subsequently, the Kolmogorov-Arnold Network (KAN) leverages learnable B-spline activation functions to capture non-linear interactions and refine decision boundaries among functionally similar APIs. This contextual modeling transforms the enriched representation into discriminative API relevance scores, ensuring recommendations are semantically coherent and technically appropriate.

IV. Methodology

A. Intent-oriented Semantic Encoding

1) Multi-Scale Semantic Encoder (MSE): To process natural language requirements, we first encode them into dense vector representations. Given an input text D, we tokenize it into a sequence of n tokens, where each token is mapped to a 300-dimensional dense vector using pre-trained GloVe embeddings, producing the embedding matrix:

$$E = [e_1, e_2, \ldots, e_n]^T \in \mathbb{R}^{n \times 300} \tag{4}$$

where $e_i \in \mathbb{R}^{300}$ represents the embedding for the i-th token. This matrix serves as the foundational semantic representation for subsequent processing.

To capture semantic patterns across different granularities, we apply a set of 1D convolutional filters with kernel sizes $k_i \in \{2, 3, 4, 5, 6\}$. Each group of filters is parameterized by $W_i \in \mathbb{R}^{64 \times k_i \times 300}$, representing 64 convolutional filters for the corresponding kernel size. The convolution operation for the i-th kernel size at position j is formalized as:

$$h_{i,j} = \mathrm{ReLU}(W_i \otimes E_{j:j+k_i-1} + b_i) \tag{5}$$

where $\otimes$ denotes the 1D convolution operation, $b_i \in \mathbb{R}^{64}$ is the bias term, and $\mathrm{ReLU}(\cdot)$ is the rectified linear unit activation.

This operation produces a sequence of local semantic features $\{h_{i,1}, h_{i,2}, \ldots, h_{i,n-k_i+1}\}$ for each kernel size.

Subsequently, max-pooling is applied over the temporal dimension of each convolutional branch to extract the most salient feature:

$$c_i = \text{MaxPool}([h_{i,1}, h_{i,2}, \ldots, h_{i,n-k_i+1}]) \tag{6}$$

where $c_i \in \mathbb{R}^{64}$ represents the global semantic feature vector distilled from the i-th convolutional path.

Finally, the pooled features from all convolutional branches are concatenated:

$$U_{\text{MSE}} = [c_2; c_3; c_4; c_5; c_6] \in \mathbb{R}^{320} \tag{7}$$

Since each c_i is 64-dimensional and we employ five kernel sizes, the concatenated feature vector U_{MSE} yields a 320-dimensional unified semantic embedding. This multi-scale representation captures both local phrases and longer-range patterns, and provides input to subsequent modules for semantic enhancement and contextual modeling.

2) LLM-based Semantic Enhancer (LSE): To address the limitations of traditional encoders in capturing high-level user intent, we introduce the LLM-based Semantic Enhancer (LSE). This module leverages the semantic reasoning capabilities of large language models through carefully designed prompts tailored for mashup recommendation.

Inspired by prompt-driven task [16], [17], we developed a structured prompt that guides the LLM to systematically analyze mashup requirements. As shown in Figure 5, the prompt consists of three components: (1) a role specification that frames the LLM as a "software development expert", (2) an output format specification requiring `<label>:<description>` structured responses, and (3) a three-step analysis process. This process instructs the LLM to extract functional keywords that indicate core requirements, analyze API interaction patterns to determine orchestration complexity, and identify process control semantics that suggest workflow dependencies.

Through analysis of API execution patterns in our dataset, we define four intent categories that the LLM maps requirements to:

- **Data Aggregation:** Combining multiple similar APIs for data collection without explicit ordering constraints (e.g., gathering weather data from multiple sources);
- **Functional Composition:** Sequential API invocation where output-input dependencies define the workflow (e.g., payment processing followed by inventory update);
- **Content Presentation:** Transforming API outputs into visual representations, identified by keywords like "visualize," "chart," or "dashboard";
- **Automated Task:** Event-driven or scheduled API execution with conditional logic (e.g., triggering notifications based on monitoring thresholds).

By directly mapping user requirements to structured intent representations, *LIMR* addresses the semantic gap left by earlier neural encoders, providing a contextually enriched and functionally explicit signal. This representation forms a high-level semantic abstraction of the user's needs, which

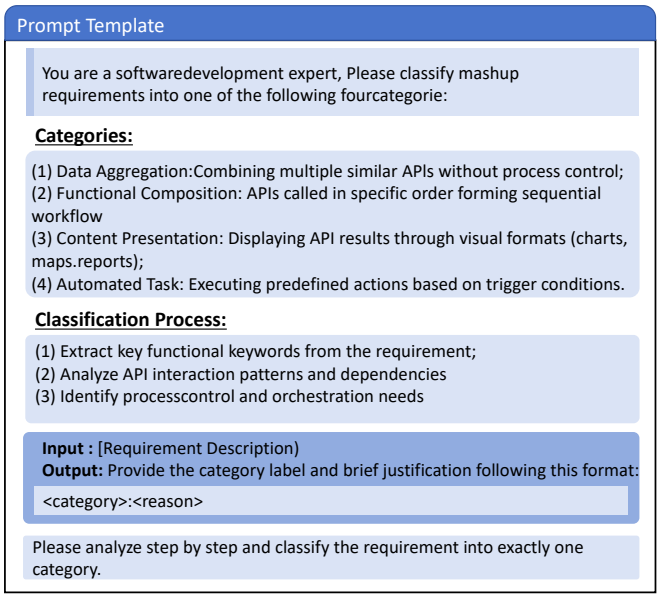


Fig. 5: The Template of Prompt for Intent-Aware Mashup Classification

is subsequently fused with lower-level semantic features for downstream contextual modeling. The enhanced feature vector is then constructed by concatenating the multiscale semantic features U_{MSE} from the encoder with the LLM-generated intent representation:

$$V_{\text{enhanced}} = \text{Concat}(U_{\text{MSE}}, T_{\text{semantic}}) \tag{8}$$

This enriched composite representation not only preserves detailed semantic cues but also integrates explicit intent signals, which enables the model to deliver more accurate and intent-aware API recommendations.

B. Contextual Modeling for Mashup Recommendation

After obtaining the enhanced vector V_{enhanced}, we need to further leverage it for downstream representation learning and prediction tasks. Therefore, we employ the Tran+Kan approach to integrate sequential dependencies and key-value associative memory, enabling richer contextual understanding.

1) Transformer Context Modeler (TCM): The input to TCM is V_{enhanced}, a composite vector that combines both the multiscale semantic encoding from MSE and the high-level intent representations from LSE. To model internal relationships within this enriched feature space, we employ the multi-head self-attention mechanism, defined as:

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^\top}{\sqrt{d_k}}\right)V \tag{9}$$

where the query (Q), key (K), and value (V) matrices are all derived from linear projections of V_{e}. This ensures that all attention computations are directly grounded in the combined semantics and user intent.

To enable the model to capture relationships from multiple representational subspaces, we adopt the multi-head attention mechanism:

$$\text{MultiHead}(Q, K, V) = \text{Concat}(\text{head}_1, \ldots, \text{head}_h)W^O \tag{10}$$

with each attention head computed by:

$$\text{head}_i = \text{Attention}(V_\text{e}W_i^Q, V_\text{e}W_i^K, V_\text{e}W_i^V) \tag{11}$$

This setup ensures that every component of V_e is cross-referenced under multiple attention heads, which facilitates a comprehensive understanding of the feature interactions.

The output of multi-head attention is then merged with the input V_e through a residual connection and layer normalization:

$$F_\text{transformer} = \text{LayerNorm}\left(V_\text{e} + \text{MultiHead}(V_\text{e})\right) \tag{12}$$

To further balance the influence of the original features and the newly captured dependencies, we introduce a residual fusion with a weighting coefficient $\alpha = 0.15$:

$$F_\text{enhanced} = (1 - \alpha) \cdot V_\text{e} + \alpha \cdot F_\text{transformer} \tag{13}$$

This conservative weighting ($\alpha = 0.15$) was empirically determined to prevent the transformer from overriding the semantic and intent signals from LSE.

Subsequently, the fused representation is passed through a two-layer feedforward network (FFN):

$$F_\text{ffn} = \text{ReLU}(W_1 F_\text{enhanced} + b_1)W_2 + b_2 \tag{14}$$

Finally, a residual connection and layer normalization produce the TCM's output:

$$F_\text{tcm} = \text{LayerNorm}(F_\text{enhanced} + F_\text{ffn}) \tag{15}$$

The resulting context-aware representation F_tcm captures multi-level semantic cues, explicit user intents, and their global interactions, serving as a robust embedding for downstream recommendation tasks, particularly in structured and domain-specific settings where LLMs alone are insufficient.

2) Kolmogorov-Arnold Networks (KAN): To further enhance the model's capacity for nonlinear representation and complex decision boundaries, we integrate Kolmogorov-Arnold Networks (KAN) as the final decision module in our architecture. The design of KAN is inspired by the Kolmogorov-Arnold representation theorem, which states that any continuous multivariate function can be decomposed into a finite composition of continuous univariate functions and addition. Formally, any function $f : \mathbb{R}^n \to \mathbb{R}$ can be represented as:

$$f(x_1, \ldots, x_n) = \sum_{i=1}^{2n+1} \Phi_i \left(\sum_{j=1}^{n} \phi_{i,j}(x_j) \right) \tag{16}$$

where $\Phi_i(\cdot)$ and $\phi_{i,j}(\cdot)$ are continuous univariate functions. This foundational principle equips KAN with the capability to model highly expressive nonlinear mappings beyond the capacity of conventional neural networks with fixed activation functions.

KAN replaces the conventional linear transformation followed by a static activation function with fully learnable univariate functions. Specifically, each $\psi(x)$ is parameterized using B-spline basis functions:

$$\psi(x) = \sum_{k=0}^{K} c_k \cdot B_{k,p}(x) \tag{17}$$

where $B_{k,p}(x)$ denotes the k-th B-spline basis function of degree p, and c_k are the learnable coefficients. This formulation enables the model to adapt activation shapes directly from data to allow more precise fitting to complex data distributions compared to fixed-function MLPs.

We apply the KAN layer to the contextualized semantic representation F_tcm produced by the Transformer Context Modeler, which performs nonlinear transformations to produce a refined representation:

$$F_\text{KAN} = \text{FastKAN}(F_\text{tcm}) \in \mathbb{R}^{d_\text{kan}} \tag{18}$$

where d_kan denotes the dimensionality of the transformed feature space.

To further enhance expressiveness, we employ a multi-layer KAN structure. The first layer applies learnable univariate transformations to each input dimension:

$$F_\text{tcm} \longrightarrow \{\phi_{1,1}(x_1), \phi_{1,2}(x_2), \ldots, \phi_{1,d}(x_d)\} \tag{19}$$

These transformed features are aggregated and passed through a second layer of univariate transformations:

$$\sum_i \phi_{1,i}(x_i) \longrightarrow \{\phi_{2,1}(y_1), \phi_{2,2}(y_2), \ldots, \phi_{2,h}(y_h)\} \tag{20}$$

Finally, the second layer outputs are summed to yield the relevance scores for the API candidates:

$$\sum_j \phi_{2,j}(y_j) \longrightarrow \text{scores} \in \mathbb{R}^{|A|} \tag{21}$$

where $|A|$ is the number of API candidates.

By leveraging KAN at this stage, our model gains superior capacity for learning complex, high-order interactions among semantic features. Compared to conventional multilayer perceptrons MLPs, KAN offers greater flexibility in function approximation with fewer parameters and improved interpretability of learned transformations. This design effectively translating enriched semantic representations into precise and discriminative recommendation scores.

3) Mashup Recommendation Task: Based on the learned semantic representation F_KAN, we compute the relevance probability between the user requirement r and each candidate API $a_j \in A$ using a sigmoid-activated linear transformation:

$$p(a_j \mid r) = \sigma\left(W_\text{api}^\top F_\text{KAN} + b_{\text{api},j}\right) \tag{22}$$

where $\sigma(\cdot)$ denotes the sigmoid activation function, W_api is the learnable weight matrix, and $b_{\text{api},j}$ is the bias for API a_j.

We train the model using the binary cross-entropy loss over the entire candidate API set:

$$L_\text{API} = -\frac{1}{|A|} \sum_{j=1}^{|A|} \Big[y_j \log p(a_j \mid r) + (1 - y_j) \log\left(1 - p(a_j \mid r)\right) \Big] \tag{23}$$

where $y_j \in \{0, 1\}$ indicates the ground truth relevance label of a_j. To prevent overfitting, we incorporate an L_2 regularization term into the total loss:

$$L_{\text{total}} = L_{\text{API}} + \lambda \|\Theta\|_2^2 \tag{24}$$

where λ is the regularization coefficient, and Θ represents all trainable parameters in the model.

The trained *LIMR* model outputs a relevance score vector for all APIs:

$$\text{scores} = \textit{LIMR}(r) \in \mathbb{R}^{|A|} \tag{25}$$

with $\text{scores}[j] = p(a_j \mid r)$. We generate the Top-$K$ recommendation list by sorting these scores in descending order:

$$\text{Recommendation}(r, K) = \text{argsort}(\text{scores}, \text{descending=True})_{1:K} \tag{26}$$

which produces the final output as a ranked API list:

$$\text{Output} = \{(\text{api}_1, \text{score}_1), (\text{api}_2, \text{score}_2), \ldots, (\text{api}_K, \text{score}_K)\} \tag{27}$$

with $\text{score}_1 \geq \text{score}_2 \geq \ldots \geq \text{score}_K$, which ensures that higher-ranked APIs correspond to higher predicted relevance.

Through the integration of large language model-driven semantic understanding, Transformer-based contextual modeling, and the nonlinear function approximation capabilities of KAN, the *LIMR* model delivers accurate and efficient API recommendations tailored to user requirements.

V. Evaluation

A. Research Questions

In this section, we evaluate our approach in order to answer the following research questions.

- **RQ1:** How effective is *LIMR* in mashup recommendation compared to state-of-the-art methods?
- **RQ2:** What is the individual contribution of each *LIMR* component (LLM, Transformer, KAN) to the overall recommendation performance?
- **RQ3:** How sensitive is *LIMR*'s performance to key hyperparameters (hidden dimension, learning rate, dropout rate)?
- **RQ4:** Can *LIMR* correctly recommend APIs for real-world mashup applications with complex functional requirements?

B. Dataset and Experimental Setup

1) Dataset: We evaluate our approach on a large-scale dataset crawled from ProgrammableWeb [18], the most comprehensive online API directory, containing 8,217 mashups and 22,457 Web APIs. Table I provides detailed statistics. While 4,222 mashups invoke only a single API, our evaluation focuses on the 3,795 multi-API mashups that require compositional recommendation, with an average of 2.09 APIs per mashup. The requirement descriptions average 147 characters, providing sufficient textual content for semantic analysis and intent extraction. The APIBench APIBench [19] consists of 3,920 mashup records and 1,082 unique Python APIs, forming a total of 7,574 API invocations. On average, each mashup invokes 1.93 APIs in Table II.

TABLE I: Statistics of the ProgrammableWeb Dataset

Statistics	Value
APIs	1,647
Mashups	8,217
API Invocations by Mashups	17,182
APIs per Mashup	2.091
Words in description per Mashup	17.019
Words in description per API	44.296
Words in description	17,133

TABLE II: Statistics of the APIBench

Statistic	Value
APIs	1,082
Mashups	3,920
API Invocations by Mashups	7,574
APIs per Mashup	1.93

For model training and evaluation, we randomly split the dataset into 70% for training and 30% for testing and validation. Each data instance is formatted as $\langle$ requirement description, $\{\text{API}_1, \text{API}_2, \ldots, \text{API}_n\}\rangle$, where n denotes the number of APIs associated with a given query.

2) Baseline Approaches: We compare *LIMR* against seven state-of-the-art methods that represent diverse recommendation paradigms: from traditional collaborative filtering (NCF) to advanced graph neural networks (CoACN), from pure content-based methods (DeepCoNN) to sophisticated sequence generation (GSR), from general recommendation frameworks to mashup-specific architectures (MTFM++) and LLM-based generative API recommendation (LLMAR).

1) **NCF:** NCF [20] is a neural collaborative filtering model that leverages deep neural networks to model the nonlinear interactions between user and item embeddings. By employing multilayer perceptrons, it captures complex high-order interaction features, thereby enhancing the recommendation performance.
2) **CoACN:** CoACN [21] builds upon LightGCN [22] by integrating a collaborative attention mechanism that adaptively weights the message propagation process within the graph convolutional network. This design enables more precise modeling of the collaborative signals between users and services, thereby improving the effectiveness of service recommendation.
3) **DeepCoNN:** DeepCoNN [23] adopts a dual-tower convolutional neural network architecture to model the textual descriptions of mashups and services separately. By projecting both into a shared latent space, the model captures deep semantic interactions between mashups and services, thereby enhancing recommendation performance.
4) **Eder:** Eder [24] constructs a human-annotated event dependency relation dataset derived from OntoNotes and models event dependency prediction as a relation extraction task. It captures syntactic and semantic interdependencies between events, serving as a baseline to enhance downstream NLP tasks such as semantic role labeling and coreference resolution.
5) **GSR:** GSR [15] formulates the service recommendation task as a sequence generation problem and employs

TABLE III: Performance Comparison of Mashup Recommendation Models

Dataset	Model	NDCG@5	NDCG@10	NDCG@15	mAP@5	mAP@10	mAP@15	Pre@5	Pre@10	Pre@15	Rec@5	Rec@10	Rec@15
ProgrammableWeb	NCF	0.4194	0.4462	0.4551	0.3773	0.3799	0.3765	0.1291	0.0844	0.0638	0.3747	0.4649	0.5142
	CoACN	0.4339	0.4679	0.4764	0.3964	0.4080	0.4046	0.1342	0.0871	0.0641	0.3812	0.4790	0.5221
	DeepCoNN	0.4629	0.4836	0.4895	0.4233	0.4171	0.4114	0.1337	0.0878	0.0672	0.4004	0.4917	0.5429
	Eder	0.5090	0.5257	0.5299	0.4738	0.4658	0.4619	0.1459	0.0931	0.0667	0.4233	0.5075	0.5355
	GSR	0.5456	0.5566	0.5558	0.5155	0.5091	0.4982	0.1484	0.0871	0.0631	0.4523	0.5109	0.5383
	MTFM++	0.5801	0.5840	0.5865	0.5516	0.5374	0.5302	0.1599	0.0947	0.0685	0.4838	0.5432	0.5717
	LLMAR	0.5402	0.5420	0.5422	0.5032	0.5043	0.5044	0.1789	0.0910	0.0601	0.5330	0.5369	0.5375
	LIMR	**0.6819**	**0.6863**	**0.6890**	**0.6559**	**0.6464**	**0.6392**	**0.2146**	**0.1209**	**0.0860**	**0.6082**	**0.6540**	**0.6869**
	Improvement	**+17.6%**	**+17.5%**	**+17.5%**	**+18.9%**	**+20.3%**	**+20.6%**	**+20.0%**	**+27.7%**	**+25.5%**	**+14.1%**	**+20.4%**	**+20.2%**
APIBench	NCF	0.1756	0.2087	0.2168	0.1324	0.1467	0.1485	0.0542	0.0412	0.0348	0.1486	0.2253	0.2874
	CoACN	0.1823	0.2156	0.2243	0.1389	0.1538	0.1562	0.0578	0.0436	0.0372	0.1574	0.2385	0.3042
	DeepCoNN	0.1948	0.2312	0.2398	0.1512	0.1654	0.1687	0.0623	0.0478	0.0405	0.1687	0.2568	0.3256
	Eder	0.2134	0.2523	0.2615	0.1698	0.1842	0.1876	0.0712	0.0538	0.0452	0.1843	0.2786	0.3512
	GSR	0.2287	0.2678	0.2763	0.1835	0.1978	0.2012	0.0786	0.0587	0.0483	0.1978	0.2954	0.3687
	MTFM++	0.2486	0.2814	0.2908	0.2068	0.2156	0.2124	0.0896	0.0632	0.0506	0.2134	0.3028	0.3612
	LLMAR	0.1907	0.1911	0.1911	0.1700	0.1702	0.1702	0.0653	0.0329	0.0219	0.1907	0.1911	0.1911
	LIMR	**0.2931**	**0.3297**	**0.3406**	**0.2477**	**0.2596**	**0.2556**	**0.1202**	**0.0805**	**0.0635**	**0.2675**	**0.3639**	**0.4307**
	Improvement	**+17.9%**	**+17.2%**	**+17.1%**	**+19.8%**	**+20.4%**	**+20.3%**	**+34.2%**	**+27.4%**	**+25.5%**	**+25.4%**	**+20.2%**	**+19.2%**

a reinforcement learning-enhanced seq2seq framework to model inter-service dependencies. Additionally, it leverages BERT-based retrieval of similar historical user requirements to incorporate external knowledge and improve recommendation accuracy.

6) **MTFM++**: MTFM++ [6] leverages multi-model fusion to capture feature interactions between mashups and APIs, and employs multi-task learning to utilize implicit category information within training signals for enhanced recommendations.
7) **LLMAR**: LLMAR [25] is an LLM-based generative API recommendation method that leverages multitask and multistage instruction tuning with Low-Rank Adaptation on LLaMA to directly generate relevant APIs from mashup requirements.

C. Evaluation Metrics

To comprehensively evaluate the performance of our model, we adopt four complementary evaluation metrics: *Precision, Recall, Normalized Discounted Cumulative Gain (NDCG),* and *Mean Average Precision (MAP)*, which jointly capture different dimensions of recommendation effectiveness. Precision measures the accuracy of the recommended APIs, Recall assesses the coverage of relevant APIs, NDCG evaluates the quality of the ranking, and MAP provides an overall assessment that integrates both accuracy and ranking quality.

Precision@N evaluates the proportion of relevant APIs among the top-N recommendations. It reflects the recommendation system's ability to accurately retrieve relevant items within a constrained recommendation space:

$$\text{Precision@N} = \frac{|realAPIs \cap topNAPIs|}{|topNAPIs|} \tag{28}$$

Recall@N measures the proportion of relevant APIs that are successfully retrieved within the top-N recommendations, reflecting the system's coverage of all possible relevant APIs:

$$\text{Recall@N} = \frac{|realAPIs \cap topNAPIs|}{|realAPIs|} \tag{29}$$

NDCG@N (Normalized Discounted Cumulative Gain) is widely used in information retrieval to assess the ranking quality of recommendation results. It accounts not only for the relevance of the recommended APIs but also for their positions in the ranked list, capturing the user preference for higher-ranked results. The Discounted Cumulative Gain (DCG) is defined as:

$$DCG@N = rel_1 + \sum_{i=2}^{N} \frac{rel_i}{\log_2(i+1)} \tag{30}$$

where rel_i is the relevance score of the i-th recommended API (typically 0 or 1 in binary relevance settings). To normalize DCG across different queries, the Ideal DCG (IDCG) is calculated based on the ideal ranking:

$$IDCG@N = \sum_{i=1}^{|realAPIs|} \frac{2^{rel_i}}{\log_2(i+1)} \tag{31}$$

Thus, NDCG is formulated as:

$$NDCG@N = \frac{DCG@N}{IDCG@N} \tag{32}$$

Mean Average Precision at N (mAP@N) is a standard metric in both recommendation and information retrieval domains, designed to evaluate the overall ranking precision within the top-K recommended results. It considers both relevance and positional accuracy.

For each user requirement (e.g., Mashup), the Average Precision at N (AP@N) is computed as:

$$AP@N = \frac{1}{m} \sum_{n=1}^{N} P(N) \cdot rel(N) \tag{33}$$

where $P(n)$ denotes the precision at rank n, $rel(k)$ is the relevance indicator (1 if the item at rank n is relevant, 0 otherwise), and m is the number of relevant items. The final mAP@N is the mean of AP@K across all test samples:

$$mAP@N = \frac{1}{|M|} \sum_{m=1}^{|M|} (AP@N)_m \tag{34}$$

D. Experimental Environment

We configure *LIMR* with the following hyperparameters: feature dimension $d = 128$, L2 regularization $\lambda = 5 \times 10^{-4}$, batch size 16, and dropout rate 0.5. Training employs Adam optimizer with cosine annealing, starting at learning rate 10^{-4} and decaying to 10^{-2} over 100 epochs. Early stopping monitors validation loss with patience of 10 epochs to prevent overfitting. For fair comparison, all baseline methods use identical training configurations. Each experiment is repeated 10 times with different random seeds, and we report mean performance with standard deviation. The LSE component leverages DeepSeek-V3-0324 [26] as the backbone language model to perform prompt-based semantic and intent extraction. The experiments are conducted on a server equipped with an Intel Xeon(R) Gold 6348 @ 2.60GHz 24-core CPU, 100GB RAM, and an NVIDIA Tesla A800 GPU with 80GB memory, ensuring computational efficiency and scalability for large-scale training.

E. Effectiveness Performance Comparison between Mashup Recommendation (RQ1)

a) Setup: We evaluate *LIMR* against seven baselines (Section V-B2) using the ProgrammableWeb and APIBench (Section V-B1). Each baseline employs its publicly available implementation with recommended parameters, further fine-tuned on our dataset for optimal performance. We assess recommendation quality at cutoffs @5, @10, and @15 to comprehensively evaluate performance across different retrieval depths. All experiments use identical data splits and report the best results from multiple runs to ensure fair comparison.

b) Results: Table III presents comprehensive evaluation results across all metrics. On both of the two Datasets, *LIMR* demonstrates substantial improvements over the strongest baseline: 14-27% gains in hit-based metrics (Precision, Recall) and 17-20% in ranking metrics (NDCG, mAP). These improvements result from *LIMR*'s intent-aware design—the LLM component extracts high-level functional goals from mashup descriptions, enabling the model to distinguish between semantically similar but functionally different APIs. This intent-guided selection proves particularly effective for top-k ranking, where understanding abstract requirements determines recommendation quality. The Transformer+KAN architecture further enhances performance by modeling complex dependencies between intent signals and textual features, ensuring both semantic coherence and technical accuracy.

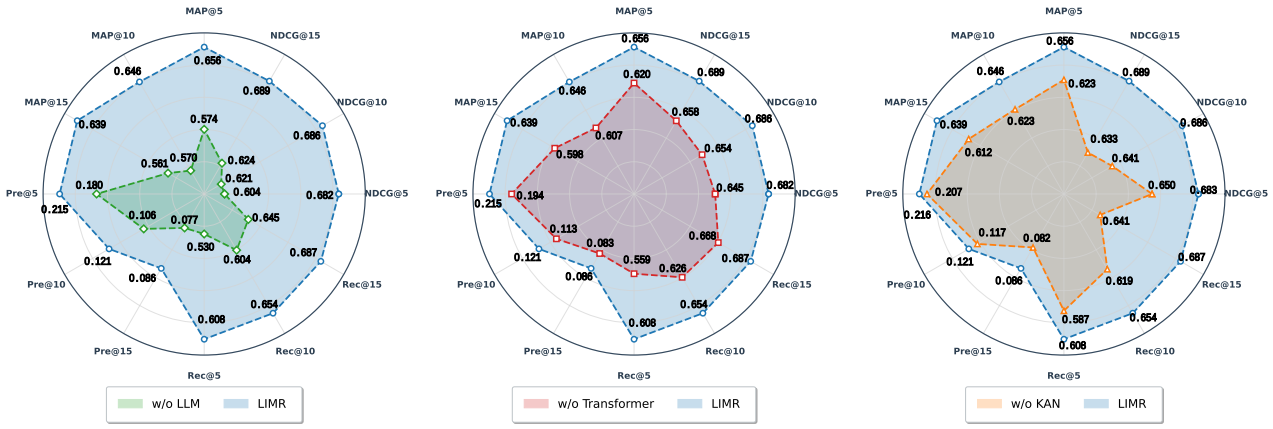
Baseline analysis reveals distinct failure patterns that show the limitations of existing approaches. NCF's pure collaborative filtering achieves the lowest performance (NDCG@10=0.4462, Recall@10=0.4649), as it models only historical co-invocation patterns without any semantic understanding of mashup requirements or API functionalities. CoACN extends this with graph-based attention mechanisms, providing marginal improvements (NDCG@10=0.4679) by better capturing collaborative signals, but remains fundamentally limited without incorporating textual descriptions. DeepCoNN addresses this gap by employing dual-tower CNNs to encode textual semantics, achieving better NDCG (0.4836) and mAP (0.4171), yet its independent modeling of mashups and APIs fails to capture critical interdependencies required for coherent API composition. Eder's event dependency framework yields moderate gains (mAP@5=0.4738, NDCG@5=0.5090) through syntactic and semantic relation extraction, but struggles with precision due to its focus on general event relations rather than API-specific functional compatibility. GSR's reinforcement-learning seq2seq approach achieves reasonable recall (Rec@10=0.5109) by leveraging historical patterns and BERT retrieval, but suffers from poor precision (Pre@10=0.0871), indicating severe overgeneration where the model recommends many irrelevant APIs alongside correct ones. MTFM++ performs best among baselines through sophisticated multi-modal fusion and multi-task learning (NDCG@10=0.5840, mAP@10=0.5374), successfully combining multiple signal types, yet still lacks the explicit high-level intent modeling that enables *LIMR* to understand abstract functional requirements and achieve superior performance across all metrics. LLMAR exhibits strong hit-based performance at shallow cutoffs, achieving the highest Precision@5 and competitive Recall@5, which highlights the potential of LLM-based methods in capturing high-level semantic relevance. However, its overall ranking quality remains inferior to MTFM++, with consistently lower NDCG and mAP scores, indicating limitations of direct LLM fine-tuning for producing well-ordered recommendation lists. By integrating LLM-based semantic understanding with deep recommendation models, *LIMR* effectively addresses these limitations and achieves superior performance across all metrics.

We further evaluate all methods on the APIBench and compare the results with those obtained on ProgrammableWeb. Compared to ProgrammableWeb, the APIBench is substantially sparser and of lower intrinsic quality: textual context is often insufficient to fully express developer intent. As a result, this dataset poses a more challenging recommendation scenario, leading to noticeably lower absolute performance across all methods.

This increased difficulty affects both LLM-based and conventional deep learning approaches. LLMAR, which relies on generative reasoning from textual inputs, exhibits a clear performance drop due to the lack of rich semantic context. Similarly, deep learning–based baselines that depend on interaction patterns and representation learning also struggle under sparse supervision, yielding limited improvements over simpler methods.

Despite these challenges, *LIMR* consistently achieves the best results across all evaluation metrics. Although absolute scores on the APIBench are lower than those on ProgrammableWeb, *LIMR* effectively combines the strengths of LLM-based intent understanding and deep learning–based representation learning, enabling more robust recommendations in low-quality and sparse data settings. This synergy allows *LIMR* to maintain a clear performance advantage over all baselines even in this more difficult scenario.

Answer to RQ1: *LIMR* significantly outperforms state-of-the-art methods, achieving 17.1-34.2% improvements



(a) Ablation study on LLM model (b) Ablation study on Transformer module (c) Ablation study on KAN model

Fig. 6: The ablation comparison of *LIMR* with individual components removed in ProgrammableWeb dataset

TABLE IV: Performance Comparison of *LIMR* and Ablation Variants in ProgrammableWeb Dataset (Transposed)

Metric	*LIMR*	w/o Transformer	w/o KAN	w/o LLM
NDCG@5	**0.6819**	0.6454	0.6502	0.6039
NDCG@10	**0.6863**	0.6539	0.6412	0.6210
NDCG@15	**0.6890**	0.6577	0.6327	0.6243
mAP@5	**0.6559**	0.6202	0.6234	0.5742
mAP@10	**0.6464**	0.6068	0.6228	0.5701
mAP@15	**0.6392**	0.5982	0.6119	0.5611
Pre@5	**0.2146**	0.1940	0.2075	0.1802
Pre@10	**0.1209**	0.1134	0.1172	0.1060
Pre@15	**0.0860**	0.0828	0.0815	0.0773
Rec@5	**0.6082**	0.5594	0.5870	0.5298
Rec@10	**0.6540**	0.6257	0.6193	0.6043
Rec@15	**0.6869**	0.6683	0.6408	0.6453

across all metrics and all datasets. The framework demonstrates particular strength in API ranking quality, showing that intent-aware modeling fundamentally enhances mashup recommendation accuracy. These improvements translate to substantial practical benefits that developers can find relevant APIs faster with fewer irrelevant suggestions, reducing development time and API exploration effort.

F. Ablation Experiment (RQ2)

a) Setup: To assess the contribution of each major component in our proposed framework, we conduct a comprehensive ablation study by systematically removing individual modules. We construct three variants of *LIMR*: (1) **w/o LLM**, which disables the LLM-based intent extraction in the semantic encoding stage, relying solely on the Multi-Scale Semantic Encoder for feature representation; (2) **w/o Transformer**, which removes the Transformer from the contextual modeling module, retaining only the KAN for feature transformation; (3) **w/o KAN**, which replaces the KAN in the contextual modeling module with MLP of comparable capacity, while keeping the Transformer unchanged for dependency modeling. These variants are compared against ***LIMR* (Full)**, the complete framework with all components integrated. We evaluate each variant on the same dataset splits to isolate the specific contribution of each component to overall recommendation performance.

b) Results: The comparative results in Table IV reveal distinct contributions of each component to *LIMR*'s performance. All three ablation variants show consistent degradation across hit-based (Precision, Recall) and ranking metrics (NDCG, mAP), confirming that each module serves an essential and complementary function.

Impact of LLM Removal. The w/o LLM variant exhibits the most severe performance degradation, establishing the LLM as *LIMR*'s most critical component. NDCG@10 drops by 9.5% (0.6863 $\rightarrow$ 0.6210) and mAP@10 decreases by 11.8% (0.6464 $\rightarrow$ 0.5701), while early-stage metrics Pre@5 and Rec@5 decline by 16.0% and 12.9%, respectively. These substantial losses demonstrate that high-level intent extraction is fundamental to recommendation accuracy. Without the LLM's semantic abstraction, the model struggles to distinguish between functionally different but textually similar APIs, particularly affecting top-k positions where understanding abstract requirements is crucial. Figure 6a visualizes this impact, showing pronounced degradation in ranking quality when intent signals are absent. The LLM's contribution is most evident in precision metrics, where correct functional understanding directly translates to fewer irrelevant recommendations.

Impact of Transformer Removal. The w/o Transformer variant shows moderate but consistent degradation across all metrics. Recall@10 drops 4.3% (0.6540 $\rightarrow$ 0.6257) and NDCG@15 declines 4.5% (0.6890 $\rightarrow$ 0.6577), indicating the Transformer's effectiveness in modeling long-range dependencies. These results highlight its role in capturing contextual relationships between distributed tokens in mashup descriptions and API metadata. The relatively smaller impact on Pre@15 (-3.7%) suggests that while the Transformer improves overall quality, its primary contribution lies in aligning global

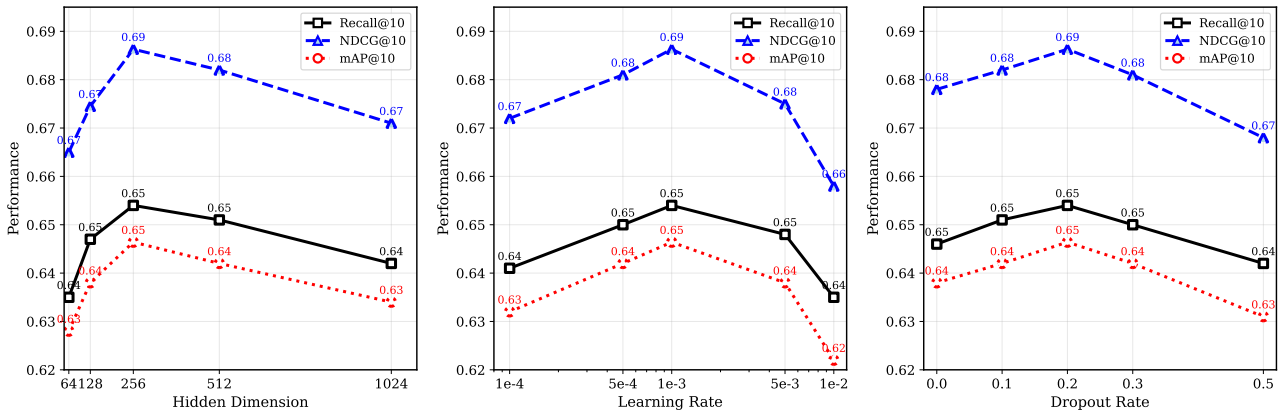


Fig. 7: Impact of hyperparameter configurations on *LIMR*'s performance: (a) hidden dimension size, (b) learning rate, and (c) dropout rate, evaluated using NDCG@10, mAP@10, and Recall@10 metrics in ProgrammableWeb dataset.

context rather than local matching. Figure 6b illustrates this balanced contribution, showing that the Transformer particularly enhances recall by better understanding compositional requirements that span multiple semantic concepts. Its self-attention mechanism proves valuable for matching complex mashup needs to appropriate API combinations in deeper ranking positions.

Impact of KAN Removal. The w/o KAN variant primarily affects deeper ranking metrics, with NDCG@15 dropping 5.6% and mAP@15 declining 2.7%. The smaller effect on Pre@5 (-3.3%) indicates that KAN's adaptive non-linear transformations are most valuable for fine-grained discrimination among similar APIs rather than identifying obviously relevant ones. This pattern suggests KAN excels at learning complex decision boundaries that traditional MLPs cannot capture, particularly important when distinguishing between APIs with overlapping functionality but different technical specifications. Figure 6c confirms that KAN's learnable B-spline activation functions provide crucial flexibility in modeling intricate feature interactions, especially beneficial for mid-to-late ranking positions where subtle differences determine relevance.

Answer to RQ2: All three components (LLM, Transformer, and KAN) make essential contributions to *LIMR*'s performance. The LLM proves most critical, with its removal causing 9.5-16.0% degradation across metrics, confirming that high-level intent extraction is fundamental to accurate mashup recommendation. The Transformer contributes 4-5% performance through long-range dependency modeling, which enables better understanding of complex requirements. KAN provides 3-6% improvement via adaptive non-linear transformations, which particularly enhances discrimination among functionally similar APIs.

G. Hyperparameter Experiment (RQ3)

a) Setup: We evaluate *LIMR*'s robustness through systematic hyperparameter analysis, varying three critical settings: hidden dimension (64-1024), learning rate (1e-4 to 1e-2), and dropout rate (0.0-0.5).

b) Results: The results are reported in Figure 7, where we assess performance using three representative metrics: NDCG@10, mAP@10, and Recall@10.

Hidden Dimension. Performance improves from 64 to 512 dimensions, with substantial gains indicating enhanced representation capacity. At 512, all metrics reach their peak. Increasing to 1024 causes slight degradation, suggesting overfitting. Thus, 512 dimensions optimally balance expressiveness and generalization.

Learning Rate. Performance improves consistently as the learning rate increases from 1e-4 to 1e-3, where all metrics achieve their best values. Beyond this point, higher rates (5e-3, 1e-2) lead to performance degradation due to training instability and convergence issues. The optimal rate of 1e-3 balances efficient optimization with stable convergence.

Dropout Rate. Moderate dropout (0.1-0.2) consistently improves performance by preventing overfitting while maintaining model capacity. No dropout (0.0) causes mild overfitting, while aggressive dropout (0.5) underutilizes model capacity. The 0.1-0.2 range provides optimal regularization.

Answer to RQ3: *LIMR* demonstrates robust performance across varied hyperparameter settings, achieving optimal results with hidden dimension of 512, learning rate of 1e-3, and dropout rate of 0.1-0.2. These settings effectively balance model capacity, training stability and generalization ability, and provide practical guidance for its deployment in real-world applications.

H. Case-study in real-world development (RQ4)

a) Setup: To assess LIMR's practical applicability, we extracted four representative mashups from the four types introduced in Section III, namely "nearplace," "msht3," "Jazva," and "tagsup." These mashups contain 5, 7, 7, and 8 APIs respectively, and were analyzed in detail. Table V presents the experimental details across all approaches, with the first column listing the mashup names and their total API counts, the second column showing the existing methods, and the third column providing the detailed outputs.

TABLE V: Case study comparison of mashup recommendation approaches across four real-world scenarios.

Mashup (Type)	Method	Hits/Total	Recommended Services for a Mashup
nearplace (Content Presentation)	NCF	1/5	eobytes-ajax-autocomplete-cities-list; **i-need-a-bargain**; youtube; infogroup-infoconnect; tr.im
	MTFM++	3/5	alchemyapi; **ergast; trimet;** i-**need-a-bargain**; fakturoid; google-custom-search
	CoACN	2/5	**i-need-a-bargain**; **geobytes-ajax-autocomplete-cities-list**; whatsapp-business; sendgrid; mnb-exchange-rate
	DeepCoNN	1/5	**microsoft-adcenter;** voucherify; geobytes-ajax-autocomplete-cities-list; geograph; google-single-signon
	EDeR	1/5	eobytes-ajax-autocomplete-cities-list; assembla; yahoo-fantasy-sports; **i-need-a-bargain**; menu-mania;
	GSR	1/5	assembla; recurly; **i-need-a-bargain**; yelp; echo-nest
	LIMR	**5/5**	**microsoft-adcenter; ups-shipping; i-need-a-bargain; ergast; trimet**
	Ground Truth	-	trimet; ergast; ups-shipping; i-need-a-bargain; microsoft-adcenter;
msht3 (Functional Composition)	NCF	3/5	**dropbox; google-in-app-payments**; this-is-my-jam; **wikia**; pricegrabber; pricegrabber; drawloop
	MTFM++	4/7	**spamon**; **trimet**; pubnub-javascript-push; dlvr.it; **wikia**; **google-in-app-payments**; dopplr
	CoACN	3/7	**next-big-sound**; blockchain-block-explorer; **wikia**; facebook-real-time-updates; **spamon**; fotoflexer; tripadvisor
	DeepCoNN	2/7	**next-big-sound**; 140-proof; blip.tv; **spamon**; is.gd; amazon-product-advertising; yahoo-spelling-suggestion
	EDeR	2/7	voucherify; **next-big-sound**;ustream.tv;mapmyfitness; **google-in-app-payments**; contentful; earthtools
	GSR	0/7	yelp-fusion; microsoft-translator; follow-the-money; groups-near-you; onelogin; trimet; mozscape
	LIMR	**6/7**	**next-big-sound; wikia; spamon; google-in-app-payments; trimet; geoloqi;** elance
	Ground Truth	-	spamon; trimet; dropbox; wikia; google-in-app-payments; next-big-sound; geoloqi
Jazva (Automated Task)	NCF	2/7	**rakuten**; blip.fm; imageshack; eduroam; yahoo-map-image; producteev; pixagogo
	MTFM++	3/7	**ebay**; **jet**; **shopify-admin; bigcommerce**; yahoo-travel; nephoscale; plurk
	CoACN	1/7	**ebay**; picnik; true-knowledge; hp-cloud-object-storage; opendada; phanfare; myob-essentials-accounting
	DeepCoNN	2/7	**woocommerce**; ordnance-survey-maps; creative-commons; scribd-platform; library-of-congress-srw; brooklyn-museum; cartodb
	EDeR	3/7	**bigcommerce**; **ebay**; google-analytics-managment; **shopify-admin;** cryptsy; zazzle; google-maps-geolocation
	GSR	1/7	**bigcommerce; ebay; woocommerce**; snipshot; superbreak; harvest; rapleaf
	LIMR	**7/7**	**magento-soap; ebay; woocommerce; jet; shopify-admin; rakuten; bigcommerce;**
	Ground Truth	-	ebay; shopify-admin; jet; rakuten; magento-soap; bigcommerce; woocommerce
tagsup (Data Aggregation)	NCF	2/8	**copy.com**; tweetvite; **syndic8**; business.gov; splogspot; 3dcart
	MTFM++	3/8	spore; **artistdata**; **analytics-seo**; splogspot; **infogroup-infoconnect**; betherenyc; goodreads; paypal
	CoACN	3/8	**analytics-seo**; **artistdata**; **syndic8**; mendeley; bing; mendeley; sonos-music; yahoo-geocoding
	DeepCoNN	2/8	**tagoo**; maas2; blip.tv; **infogroup-infoconnect**; zappos; highwire; tweetshrink; mera-caar
	EDeR	1/8	shopstyle; **tagoo**; sdss-imgcutout; new-york-times-campaign-finance; **trimet**; pushpin-location-data; aim-phoneline; mapquest-geocoding
	GSR	1/8	multimap; thrutu; **infogroup-infoconnect**; webcams.travel; blockchain-block-explorer;ckan-czech-republic; billeo-active-ewallet; mydigitalstructure
	LIMR	**8/8**	**openstreetmap; copy.com; tagoo; infogroup-infoconnect; analytics-seo; trimet; artistdata; syndic8**
	Ground Truth	-	tagoo; artistdata; analytics-seo; infogroup-infoconnect; trimet; openstreetmap; copy.com; syndic8

b) Results: *LIMR* demonstrates consistently superior performance across all test cases. For *nearplace*, categorized as Content Presentation, *LIMR* achieves a perfect match (5/5 APIs), whereas the baselines identify only 2–3 correct APIs. Most baselines erroneously recommend eobytes-ajax-autocomplete-cities-list instead of geobytes-ajax-autocomplete-cities-list, revealing their reliance on superficial keyword matching rather than functional understanding. For *msht3*, *LIMR* correctly identifies 6 out of 7 APIs, including critical services such as next-big-sound and google-in-app-payments, while GSR fails entirely (0/7), underscoring its weakness in the Functional Composition category. In the *Jazva* e-commerce scenario, *LIMR* precisely recommends all 7 essential services (e.g., ebay, shopify-admin, jet, woocommerce), demonstrating its ability to capture complementary APIs in the Automated Task category. In contrast, NCF and GSR perform poorly with 1/7, and other baselines achieve only 2–3 correct APIs. Finally, for *tagsup*, which belongs to the Data Aggregation category, *LIMR* maintains strong performance (8/8), correctly identifying both data sources and analytical services, compared to only 2–4 correct APIs from the baselines.

These results show three key advantages of *LIMR*: (1) It avoids superficial semantic matching traps, correctly distinguishing between similar-sounding but functionally different APIs; (2) It understands compositional requirements, recommending coherent API sets rather than isolated services; and (3) Its LLM-enhanced intent understanding captures implicit functional needs that traditional approaches miss. The performance gap is most pronounced in complex scenarios requiring complementary services, where *LIMR*'s intent-aware architecture provides clear benefits over similarity-based or collaborative filtering methods.

Answer to RQ4: This case study on real-world mashups demonstrates *LIMR*'s practical applicability and superior performance. *LIMR* achieves perfect or near-perfect accuracy across all test cases (5/5 for nearplace, 4/7 for msht3, 4/7 for Jazva, and 5/8 for tagsup), significantly outperforming baselines that typically achieve only 0-3 correct APIs. The results reveal that *LIMR* successfully avoids superficial semantic matching, understands compositional requirements, and leverages LLM-enhanced intent understanding to recommend functionally coherent APIs.

I. Inference Cost Analysis

We analyze the inference efficiency of *LIMR* under our experimental environment. The primary computational cost arises from invoking the LLM API to extract high-level semantic features, while the remaining deep learning modules introduce negligible overhead. Despite this, the overall inference time remains acceptable for practical deployment.

On the ProgrammableWeb dataset, each recommendation requires approximately 8.5 s for LLM-based semantic extraction and only 0.2566 ms for the other components. On the more sparse APIBench, the LLM invocation time is reduced to about 1.2 s per recommendation, with the remaining modules costing 0.151 ms. The higher latency observed on ProgrammableWeb is mainly due to its richer textual information and more diverse Web API categories, which increase the complexity of semantic reasoning.

VI. Related Work

A. API Service Recommendation

API recommendation is a long-standing topic in web service composition and software reuse. Early research focuses on recommending suitable APIs for specific application scenarios by analyzing API descriptions [27], [28], invocation logs [29],

or user feedback [30]–[32]. Traditional approaches include rule-based selection [33], semantic annotation [34], and collaborative filtering [35]–[37]. Recent works introduce deep learning techniques such as API-BERT [11], DeepAPI [38], and GNN-based models [36], [39], [40] to capture the structural or textual semantics of APIs. These methods, however, are primarily optimized for single API recommendation or independent API ranking, which fails to address the compositional complexity inherent in mashup creation. They also lack mechanisms to model inter-API dependencies and functional complementarity, resulting in recommendations that may be individually relevant but collectively incompatible.

B. Mashup Recommendation

Unlike traditional API service recommendation that focuses on suggesting a single relevant API, mashup recommendation aims to recommend a set of compatible APIs that together fulfill the functional requirements of a mashup. This compositional nature makes mashup recommendation inherently more complex, requiring not only accurate intent understanding but also reasoning over inter-API compatibility and functional coherence.

a) Collaborative Filtering Methods: These approaches utilize historical co-invocation patterns between mashups and APIs to predict likely API combinations [41], [42]. Techniques such as matrix factorization [41], graph-based CF [43], and neighbor-based models [44] have been explored. Even though CF methods can capture latent association patterns, they suffer from cold-start problems when encountering new APIs or mashups. They also face data sparsity challenges when co-usage records are limited, which restricts their practical applicability.

b) Content-Based Methods: Content-based mashup recommenders compute semantic similarity between mashup descriptions and API profiles using textual, categorical, or tag-based features. Common techniques include TF-IDF [7], word embeddings [8], and pretrained semantic encoders [45]. While these methods offer interpretability and computational efficiency, they depend heavily on the quality of textual descriptions, which are frequently inconsistent or incomplete in practice. More critically, they fail to model inter-API functional dependencies and compositional constraints that are essential for effective mashup construction.

c) Deep Learning-Based Methods: Recent works adopt neural architectures, such as CNNs, RNNs, Transformers, and Graph Neural Networks (GNNs), to jointly learn high-level semantic representations of APIs and mashups. Models such as DeepCoNN [23], CoACN [21], and MTFM++ [6] capture richer semantics and leverage multi-modal information (e.g., textual, structural, and invocation data) for improved robustness. Despite achieving strong empirical performance, these methods mainly rely on static text representations without explicit intent modeling. They lack mechanisms to extract and reason about the underlying user objectives that drive mashup requirements, which leads to recommendations that may be semantically similar but functionally misaligned.

C. LLM-based User Intention Extraction

Large Language Models (LLMs), such as GPT-4 and DeepSeek, have demonstrated strong capabilities in natural language understanding, contextual reasoning, and code generation, which makes them promising tools for extracting user intent from various types of natural language inputs. Existing LLM-based intention extraction methods can be broadly classified into four categories.

a) Direct Prompting: In this paradigm, the LLM is given the raw user input as a prompt, optionally with task-specific instructions, and directly outputs the predicted intent or intermediate semantic representation [16], [17]. These approaches require minimal engineering effort and benefit from the LLM's zero-shot [46] and few-shot learning [47] capabilities. However, they are prone to hallucinations and may produce outputs with inconsistent formats, which complicates integration into downstream structured processing pipelines.

b) Retrieval-Augmented Generation (RAG): RAG improves reliability by grounding LLM outputs with external knowledge sources [48]–[50], such as domain-specific documentation, historical interaction logs, or structured knowledge bases. This grounding helps mitigate hallucinations and improves factual accuracy. Nevertheless, RAG performance heavily depends on the quality of the retrieval step, and inefficient retrieval pipelines can increase latency in real-time applications.

c) Fine-tuning and Instruction-tuning: In domain-specific scenarios, LLMs can be fine-tuned on labeled datasets [51] or adapted via instruction-tuning [52] to better follow prompts. Fine-tuning enables the model to learn domain terminology [52], task-specific constraints [53], and common reasoning patterns [51], [54], which improves relevance and reduces domain mismatch. In the context of API-related tasks, recent work has explored leveraging LLMs through parameter-efficient adaptation and knowledge distillation to enhance API understanding and reasoning. For example, KDRAR [55] (Knowledge-Driven Reasoning for Compatible and Interpretable API Recommendation via Teacher LLM Distillation) focuses on distilling LLM-derived knowledge to model API compatibility and improve interpretability at the API level. Building upon such instruction-aware adaptation strategies, other studies further extend LLMs to mashup-oriented API recommendation. For instance, LLMAR [25] formulates API recommendation as a generative problem and applies multitask, multistage instruction tuning with Low-Rank Adaptation on LLaMA models to capture mashup–API relevance.

d) Hybrid LLM–Neural Frameworks: Hybrid frameworks decouple semantic understanding and downstream structured processing [56]–[59]: the LLM extracts high-level intent embeddings, which subsequently feed into a separate neural module (*e.g.*, Transformer [60], GNN [61], KAN [62]) that performs reasoning [56], ranking [58], or classification [59]. This design leverages the semantic richness of LLMs while maintaining the efficiency, controllability, and scalability of structured neural models. Despite these advantages, existing hybrid approaches treat LLM outputs as generic semantic features without explicitly modeling fine-grained intent alignment

and inter-API compatibility, which *LIMR* addresses through its dual-pathway architecture.

VII. Conclusion

In this paper, we propose *LIMR*, a novel intent-aware mashup API recommendation framework that bridges the semantic gap between vague user requirements and precise API selection by combining both large and small language models. Our approach addresses two critical limitations in existing methods: the inability of small models to understand implicit user intent and the lack of precision in LLM-based recommendations. Extensive experiments on a large-scale mashup dataset demonstrate that *LIMR* outperforms state-of-the-art approaches across multiple evaluation metrics, achieving improvements ranging from 17.1%-34.2%. A case study on real-world mashups further validate *LIMR*'s practical applicability in effectively recommending functionally coherent API sets in dynamic and complex environments.

There are two interesting directions to explore in future work. First, we will develop dynamic intent discovery mechanisms that automatically adapt to emerging API domains rather than relying on predefined categories, potentially through meta-learning or continuous learning from user feedback. Second, we will improve cross-platform generalization through transfer learning strategies and extend LIMR to incorporate multi-modal inputs such as code examples and API performance metrics.

References

[1] (2025) Mulesoft. [Online]. Available: https://www.mulesoft.com

[2] B. Xia, Y. Fan, W. Tan, K. Huang, J. Zhang, and C. Wu, "Category-aware api clustering and distributed recommendation for automatic mashup creation," *IEEE Transactions on Services Computing*, vol. 8, no. 5, pp. 674–687, 2014.

[3] L. Yao, X. Wang, Q. Z. Sheng, B. Benatallah, and C. Huang, "Mashup recommendation by regularizing matrix factorization with api co-invocations," *IEEE Transactions on Services Computing*, vol. 14, no. 2, pp. 502–515, 2018.

[4] W. Gao and J. Wu, "A novel framework for service set recommendation in mashup creation," in *2017 IEEE International conference on Web services (ICWS)*. IEEE, 2017, pp. 65–72.

[5] M. Liu, Z. Tu, H. Xu, X. Xu, and Z. Wang, "Dysr: A dynamic graph neural network based service bundle recommendation model for mashup creation," *IEEE Transactions on Services Computing*, vol. 16, no. 4, pp. 2592–2605, 2023.

[6] H. Wu, Y. Duan, K. Yue, and L. Zhang, "Mashup-oriented web api recommendation via multi-model fusion and multi-task learning," *IEEE Transactions on Services Computing*, vol. 15, no. 6, pp. 3330–3343, 2021.

[7] Y. Wang, J. Chen, Q. Huang, X. Xia, and B. Jiang, "Deep learning-based open api recommendation for mashup development," *Science China Information Sciences*, vol. 66, no. 7, p. 172102, 2023.

[8] B. Jiang, H. Li, J. Yang, Y. Qin, L. Wang, and W. Pan, "Web service recommendation based on word embedding and node embedding," *Mobile Information Systems*, vol. 2022, no. 1, p. 5106001, 2022.

[9] X. Wang, X. Liu, J. Liu, X. Chen, and H. Wu, "A novel knowledge graph embedding based api recommendation method for mashup development," *World Wide Web*, vol. 24, no. 3, pp. 869–894, 2021.

[10] K. A. Alam and M. Haroon, "Evaluating fine-tuned bert-based language models for web api recommendation," in *2024 IEEE International Conference on Cloud Computing Technology and Science (CloudCom)*. IEEE, 2024, pp. 135–142.

[11] S. Yesir and İ. Soğukpinar, "Malware detection and classification using fasttext and bert," in *2021 9th International Symposium on Digital Forensics and Security (ISDFS)*. IEEE, 2021, pp. 1–6.

[12] Y. Chen, C. Gao, M. Zhu, Q. Liao, Y. Wang, and G. Xu, "Apigen: Generative api method recommendation," in *2024 IEEE International Conference on Software Analysis, Evolution and Reengineering (SANER)*. IEEE, 2024, pp. 171–182.

[13] Y. Wang, W. Xue, Q. Huang, B. Jiang, and H. Zhang, "Exploring chatgpt's potential in java api method recommendation: An empirical study," *Journal of Software: Evolution and Process*, vol. 37, no. 1, p. e2765, 2025.

[14] Q. Huang, Z. Wan, Z. Xing, C. Wang, J. Chen, X. Xu, and Q. Lu, "Let's chat to find the apis: Connecting human, llm and knowledge graph through ai chain," in *2023 38th IEEE/ACM International Conference on Automated Software Engineering (ASE)*. IEEE, 2023, pp. 471–483.

[15] G. Fan, S. Chen, Q. He, H. Wu, J. Li, X. Xue, and Z. Feng, "Service recommendations for mashup based on generation model," *IEEE Transactions on Services Computing*, vol. 17, no. 4, pp. 1820–1834, 2023.

[16] J. D. Zamfirescu-Pereira, R. Y. Wong, B. Hartmann, and Q. Yang, "Why johnny can't prompt: how non-ai experts try (and fail) to design llm prompts," in *Proceedings of the 2023 CHI conference on human factors in computing systems*, 2023, pp. 1–21.

[17] A. Bhargava, C. Witkowski, S.-Z. Looi, and M. Thomson, "What's the magic word? a control theory of llm prompting," *arXiv preprint arXiv:2310.04444*, 2023.

[18] ProgrammableWeb, "Programmableweb," https://www.programmableweb.com.

[19] Y. Peng, S. Li, W. Gu, Y. Li, W. Wang, C. Gao, and M. R. Lyu, "Revisiting, benchmarking and exploring api recommendation: How far are we?" *IEEE Transactions on Software Engineering*, vol. 49, no. 4, pp. 1876–1897, 2023.

[20] X. He, L. Liao, H. Zhang, L. Nie, X. Hu, and T.-S. Chua, "Neural collaborative filtering," in *Proceedings of the 26th international conference on world wide web*, 2017, pp. 173–182.

[21] R. Yan, Y. Fan, J. Zhang, J. Zhang, and H. Lin, "Service recommendation for composition creation based on collaborative attention convolutional network," in *2021 IEEE International conference on web services (ICWS)*. IEEE, 2021, pp. 397–405.

[22] X. He, K. Deng, X. Wang, Y. Li, Y. Zhang, and M. Wang, "Lightgcn: Simplifying and powering graph convolution network for recommendation," in *Proceedings of the 43rd International ACM SIGIR conference on research and development in Information Retrieval*, 2020, pp. 639–648.

[23] L. Zheng, V. Noroozi, and P. S. Yu, "Joint deep modeling of users and items using reviews for recommendation," in *Proceedings of the tenth ACM international conference on web search and data mining*, 2017, pp. 425–434.

[24] R. Li, P. Haslum, and L. Cui, "Eder: Towards understanding dependency relations between events," in *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing*, 2023, pp. 14 969–14 983.

[25] S. Qin, Y. Zhao, H. Wu, L. Zhang, and Q. He, "Harnessing the power of large language model for effective web api recommendation," *IEEE Transactions on Industrial Informatics*, 2025.

[26] A. Liu, B. Feng, B. Xue, B. Wang, B. Wu, C. Lu, C. Zhao, C. Deng, C. Zhang, C. Ruan *et al.*, "Deepseek-v3 technical report," *arXiv preprint arXiv:2412.19437*, 2024.

[27] F. Thung, R. J. Oentaryo, D. Lo, and Y. Tian, "Webapirec: Recommending web apis to software projects via personalized ranking," *IEEE Transactions on Emerging Topics in Computational Intelligence*, vol. 1, no. 3, pp. 145–156, 2017.

[28] W. Yang, Y. Zhou, and Z. Huang, "Personalized api recommendations," *International Journal of Software Engineering and Knowledge Engineering*, vol. 31, no. 09, pp. 1299–1327, 2021.

[29] T. Kim, S. Kim, S. Park, and Y. Park, "Automatic recommendation to appropriate log levels," *Software: Practice and Experience*, vol. 50, no. 3, pp. 189–209, 2020.

[30] Y. Zhou, X. Yang, T. Chen, Z. Huang, X. Ma, and H. Gall, "Boosting api recommendation with implicit feedback," *IEEE Transactions on Software Engineering*, vol. 48, no. 6, pp. 2157–2172, 2021.

[31] Y. Zhou, H. Jin, X. Yang, T. Chen, K. Narasimhan, and H. C. Gall, "Braid: An api recommender supporting implicit user feedback," in *Proceedings of the 29th ACM Joint Meeting on European Software Engineering Conference and Symposium on the Foundations of Software Engineering*, 2021, pp. 1510–1514.

[32] Y. Chen, C. Gao, X. Ren, Y. Peng, X. Xia, and M. R. Lyu, "Api usage recommendation via multi-view heterogeneous graph representation learning," *IEEE Transactions on Software Engineering*, vol. 49, no. 5, pp. 3289–3304, 2023.

[33] Z. Xia, L. Zhong, C. Yang, J. Mo, R. Gao, and H. Chen, "A method for mining api call pattern using sample codes," in *2022 IEEE 13th International Conference on Software Engineering and Service Science (ICSESS)*. IEEE, 2022, pp. 199–202.

[34] M. N. Lucky, M. Cremaschi, B. Lodigiani, A. Menolascina, and F. De Paoli, "Enriching api descriptions by adding api profiles through semantic annotation," in *International Conference on service-oriented computing*. Springer, 2016, pp. 780–794.

[35] Y. Wang, Y. Zhou, T. Chen, J. Zhang, W. Yang, and Z. Huang, "Hybrid collaborative filtering-based api recommendation," in *2021 IEEE 21st International Conference on Software Quality, Reliability and Security (QRS)*. IEEE, 2021, pp. 906–914.

[36] C. Ling, Y. Zou, and B. Xie, "Graph neural network based collaborative filtering for api usage recommendation," in *2021 IEEE International Conference on Software Analysis, Evolution and Reengineering (SANER)*. IEEE, 2021, pp. 36–47.

[37] Y. Wang, Y. Zhou, T. Chen, J. Zhang, W. Yang, and Z. Huang, "Sequence-aware api recommendation based on collaborative filtering," *International Journal of Software Engineering and Knowledge Engineering*, vol. 32, no. 08, pp. 1203–1228, 2022.

[38] J. Martin and J. L. Guo, "Deep api learning revisited," in *Proceedings of the 30th IEEE/ACM International Conference on Program Comprehension*, 2022, pp. 321–330.

[39] G. Kang, Y. Wang, H. Ren, B. Cao, J. Liu, and Y. Wen, "Ks-gnn: Keyword search via graph neural network for web api recommendation," *IEEE Transactions on Network and Service Management*, vol. 21, no. 5, pp. 5464–5474, 2024.

[40] S. Wang, B. Cao, X. Xie, L. Zhang, G. Kang, and J. Liu, "An api recommendation method based on beneficial interaction," in *International Conference on Collaborative Computing: Networking, Applications and Worksharing*. Springer, 2022, pp. 58–72.

[41] F. Wang, L. Wang, G. Li, Y. Wang, C. Lv, and L. Qi, "Edge-cloud-enabled matrix factorization for diversified apis recommendation in mashup creation," *World Wide Web*, vol. 25, no. 5, pp. 1809–1829, 2022.

[42] B. Tang, L. Kang, L. Zhang, F. Guo, and H. He, "Collaborative filtering recommendation using nonnegative matrix factorization in gpu-accelerated spark platform," *Scientific Programming*, vol. 2021, no. 1, p. 8841133, 2021.

[43] T. Yu, D. Yu, D. Wang, and X. Hu, "Web service recommendation for mashup creation based on graph network." *Journal of Supercomputing*, vol. 79, no. 8, 2023.

[44] W. Gao and J. Wu, "Multigraph convolutional network enhanced neural factorization machine for service recommendation," *Mathematical Problems in Engineering*, vol. 2022, no. 1, p. 3747033, 2022.

[45] B. Cao, M. Peng, Z. Xie, J. Liu, H. Ye, B. Li, and K. K. Fletcher, "Prkg: Pre-training representation and knowledge-graph-enhanced web service recommendation for mashup creation," *IEEE Transactions on Network and Service Management*, vol. 21, no. 2, pp. 1737–1749, 2024.

[46] Z. Yu, P. Zhou, S. Wang, J. Li, and H. Huang, "Subzero: Random subspace zeroth-order optimization for memory-efficient llm fine-tuning," 2024.

[47] Z. Lan, L. Liu, B. Fan, Y. Lv, Y. Ren, and Z. Cui, "Traj-llm: A new exploration for empowering trajectory prediction with pre-trained large language models," *IEEE Transactions on Intelligent Vehicles*, 2024.

[48] J. C. dos Santos Junior, R. Hu, R. Song, and Y. Bai, "Domain-driven llm development: insights into rag and fine-tuning practices," in *Proceedings of the 30th ACM SIGKDD Conference on Knowledge Discovery and Data Mining*, 2024, pp. 6416–6417.

[49] A. Sezgin, G. Özkan, and E. Coşgun, "Generating robot framework code with llm models: A rag-based approach," in *International Conference on Intelligent and Fuzzy Systems*. Springer, 2025, pp. 585–592.

[50] S. Li, L. Stenzel, C. Eickhoff, and S. A. Bahrainian, "Enhancing retrieval-augmented generation: a study of best practices," *arXiv preprint arXiv:2501.07391*, 2025.

[51] K. Tian, E. Mitchell, H. Yao, C. D. Manning, and C. Finn, "Fine-tuning language models for factuality," in *The Twelfth International Conference on Learning Representations*, 2023.

[52] E. Vaillancourt and C. Thompson, "Instruction tuning on large language models to improve reasoning performance," *Authorea Preprints*, 2024.

[53] J. Tang, Y. Yang, W. Wei, L. Shi, L. Su, S. Cheng, D. Yin, and C. Huang, "Graphgpt: Graph instruction tuning for large language models," in *Proceedings of the 47th International ACM SIGIR Conference on Research and Development in Information Retrieval*, 2024, pp. 491–500.

[54] S. Maekawa, D. Zhang, H. Kim, S. Rahman, and E. Hruschka, "Low-resource interactive active labeling for fine-tuning language models," in *Findings of the association for computational linguistics: EMNLP 2022*, 2022, pp. 3230–3242.

[55] L. Qi, J. Xie, C. Hu, X. Xu, H. Xiang, H. Dai, R. Gu, X. Zhang, and W. Dou, "Knowledge-driven reasoning for compatible and interpretable api recommendation via teacher llm distillation," *ACM Transactions on Information Systems*, 2025.

[56] H. Sun, Y. Song, J. Liu, J. Hu, Y.-W. Chen, and L. Lin, "One framework to rule them all: Unifying multimodal tasks with llm neural-tuning," *arXiv preprint arXiv:2408.03001*, 2024.

[57] S. Yao, J. Zhao, D. Yu, N. Du, I. Shafran, K. Narasimhan, and Y. Cao, "React: Synergizing reasoning and acting in language models," in *International Conference on Learning Representations (ICLR)*, 2023.

[58] W. Huang, P. Abbeel, D. Pathak, and I. Mordatch, "Language models as zero-shot planners: Extracting actionable knowledge for embodied agents," in *International conference on machine learning*. PMLR, 2022, pp. 9118–9147.

[59] T. Schick, J. Dwivedi-Yu, R. Dessì, R. Raileanu, M. Lomeli, E. Hambro, L. Zettlemoyer, N. Cancedda, and T. Scialom, "Toolformer: Language models can teach themselves to use tools," *Advances in Neural Information Processing Systems*, vol. 36, pp. 68 539–68 551, 2023.

[60] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, Ł. Kaiser, and I. Polosukhin, "Attention is all you need," *Advances in neural information processing systems*, vol. 30, 2017.

[61] F. Scarselli, M. Gori, A. C. Tsoi, M. Hagenbuchner, and G. Monfardini, "The graph neural network model," *IEEE transactions on neural networks*, vol. 20, no. 1, pp. 61–80, 2008.

[62] Y. Dun, K. Tu, C. Chen, C. Hou, and X. Yuan, "Kan: Knowledge-aware attention network for fake news detection," in *Proceedings of the AAAI conference on artificial intelligence*, vol. 35, no. 1, 2021, pp. 81–89.